

深圳大学实验报告

课程名称： 计算机网络

实验项目名称： 常用的网络命令

学院： 计算机与软件学院

专业： 计算机科学与技术

指导教师： 谢瑞桃

报告人： 王云舒 学号： 2018152044 班级： 计科 03

实验时间： 2021/3/15

实验报告提交时间： 2021/3/23

教务处制

实验目的：

了解 *ping*、*ipconfig*、*netstat*、*tracert*、*ARP*、*route*、*nslookup* 等常用网络工具的功能以及使用方法，并通过这些工具发现或者验证网络中的故障。

实验环境：

具有 *Internet* 连接的 *Windows 10* 操作系统；*Windows PowerShell*。

实验内容：

使用以下七种网络调试工具分析网络情况。

1. *ipconfig*
2. *ping*
3. *netstat*
4. *tracert*
5. *ARP*
6. *nslookup*
7. *route*

要求参考讲义学习七种网络调试工具，理解每种工具的用途以及使用方法，并使用每种工具的各种指令依照步骤完成实验内容 1-7。最后对实验结果截图并撰写实验报告。

实验步骤：

1 *ipconfig*

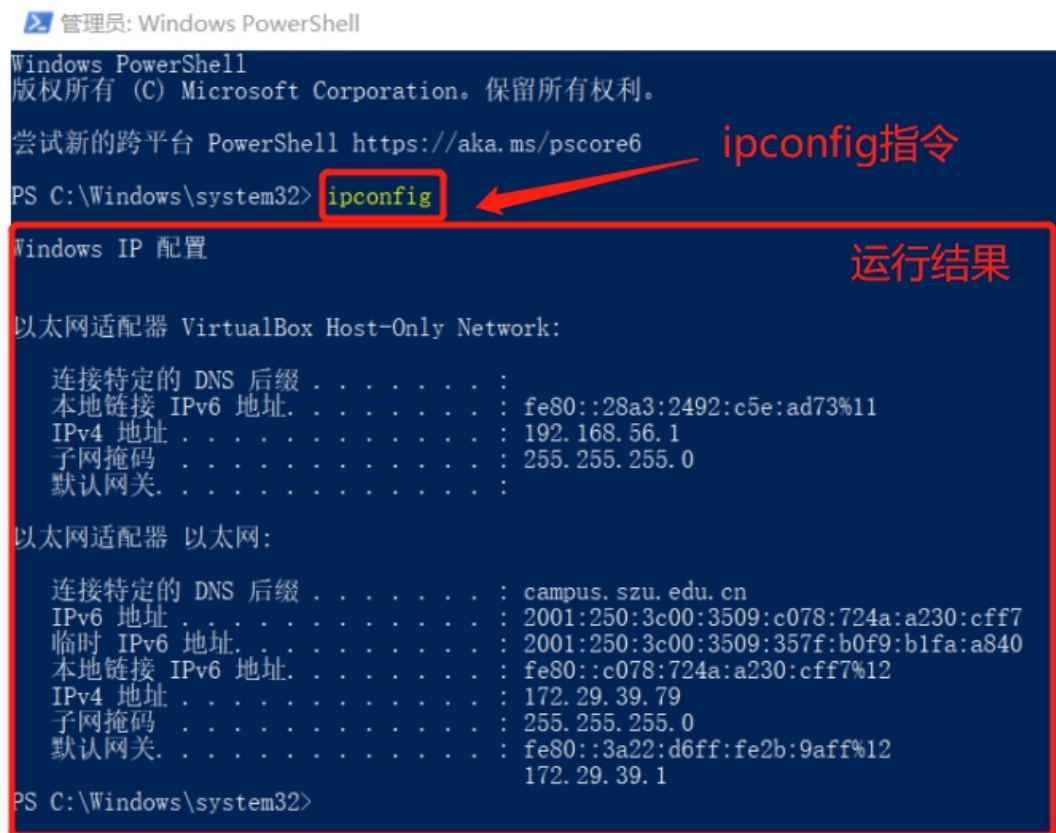
ipconfig 指令可用于显示主机当前的 **IPv6 地址**、**IPv4 地址**、**子网掩码**和**默认网关**。其中，子网掩码是用于将主机（*host*）的 *IP* 地址分解成网络地址（*network address*）和主机地址（*host address*）的一组编码。形式上，子网掩码是 32 位的，比如 255.255.255.0，一共是四个 8 位组，即 11111111.11111111.11111111.00000000。将这个子网掩码与主机的 *IP* 地址（192.168.16.187）进行二进制与运算，就可获得其网络地址，其结果就是 192.168.16.0。默认网关（*Default Gateway*）就是电脑要向网络上发送数据包时默认发送的地方。从网络架构来看，默认网关就是跟电脑在同一个网段里面（一般就是所在的子网），

负责与其它网段（子网）进行通讯的那台设备。

本次实验在 *Windows 10* 操作系统下进行，右击开始菜单选择“*Windows Powershell*（管理员）”即可打开 *PowerShell*。

1.1 *ipconfig*（不带参数）

当不带任何参数选项使用时，该指令显示每个接口的 *IP* 地址、子网掩码和默认网关。在 *PowerShell* 内输入该指令，得到结果如图 1-1 所示。可以看到，本机 *IP* 为 **172.29.39.79**，网关 *IP* 为 **172.29.39.1**。除了以太网外的另一个接口是电脑上所装的虚拟机，可以看到两个 *IP* 地址是不同的。



```
管理员: Windows PowerShell
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。

尝试新的跨平台 PowerShell https://aka.ms/pscore6

PS C:\Windows\system32> ipconfig

Windows IP 配置

以太网适配器 VirtualBox Host-Only Network:

    连接特定的 DNS 后缀 . . . . . :
    本地链接 IPv6 地址. . . . . : fe80::28a3:2492:c5e:ad73%11
    IPv4 地址 . . . . . : 192.168.56.1
    子网掩码 . . . . . : 255.255.255.0
    默认网关. . . . . :

以太网适配器 以太网:

    连接特定的 DNS 后缀 . . . . . : campus.szu.edu.cn
    IPv6 地址 . . . . . : 2001:250:3c00:3509:c078:724a:a230:cff7
    临时 IPv6 地址. . . . . : 2001:250:3c00:3509:357f:b0f9:b1fa:a840
    本地链接 IPv6 地址. . . . . : fe80::c078:724a:a230:cff7%12
    IPv4 地址 . . . . . : 172.29.39.79
    子网掩码 . . . . . : 255.255.255.0
    默认网关. . . . . : fe80::3a22:d6ff:fe2b:9aff%12
                        172.29.39.1

PS C:\Windows\system32>
```

图 1-1 *ipconfig*（不带参数）的运行结果

1.2 *ipconfig /all*

当使用 *all* 选项时，显示完整配置信息，包括 *DNS* 服务器、*DHCP* 服务器、*IP* 地址获得租约的时间、*IP* 地址租约过期的时间等。得到结果如图 1-2 所示。该条指令需要注意 */all* 与 *ipconfig* 必须间隔空格，否则 *PowerShell* 无法识别该指令。

```
管理员: Windows PowerShell
PS C:\Windows\system32> ipconfig /all

Windows IP 配置

主机名 . . . . . : DESKTOP-BKQJ7B9
主 DNS 后缀 . . . . . :
节点类型 . . . . . : 混合
IP 路由已启用 . . . . . : 否
WINS 代理已启用 . . . . . : 否
DNS 后缀搜索列表 . . . . . : campus.szu.edu.cn

以太网适配器 VirtualBox Host-Only Network: ①虚拟机的网络配置

连接特定的 DNS 后缀 . . . . . :
描述. . . . . : VirtualBox Host-Only Ethernet Adapter
物理地址. . . . . : 0A-00-27-00-00-0B
DHCP 已启用 . . . . . : 否
自动配置已启用. . . . . : 是
本地链接 IPv6 地址. . . . . : fe80::28a3:2492:c5e:ad73%11(首选)
IPv4 地址 . . . . . : 192.168.56.1(首选)
子网掩码 . . . . . : 255.255.255.0
默认网关. . . . . :
DHCPv6 IAID . . . . . : 336199719
DHCPv6 客户端 DUID . . . . . : 00-01-00-01-27-CE-C5-04-A8-5E-45-C0-E9-BC
DNS 服务器 . . . . . : fec0:0:0:ffff::1%1
                          fec0:0:0:ffff::2%1
                          fec0:0:0:ffff::3%1
TCP/IP 上的 NetBIOS . . . . . : 已启用

以太网适配器 以太网: ②本机以太网的网络配置

连接特定的 DNS 后缀 . . . . . : campus.szu.edu.cn
描述. . . . . : Realtek PCIe GbE Family Controller
物理地址. . . . . : A8-5E-45-C0-E9-BC
DHCP 已启用 . . . . . : 是
自动配置已启用. . . . . : 是
IPv6 地址 . . . . . : 2001:250:3c00:3509:c078:724a:a230:cff7(首选)
临时 IPv6 地址. . . . . : 2001:250:3c00:3509:357f:b0f9:b1fa:a840(首选)
本地链接 IPv6 地址. . . . . : fe80::c078:724a:a230:cff7%12(首选)
IPv4 地址 . . . . . : 172.29.39.79(首选)
子网掩码 . . . . . : 255.255.255.0
获得租约的时间 . . . . . : 2021年3月18日 19:22:03
租约过期的时间 . . . . . : 2021年3月18日 22:52:06
默认网关. . . . . : fe80::3a22:d6ff:fe2b:9aff%12
                          172.29.39.1
DHCP 服务器 . . . . . : 172.30.254.81
DHCPv6 IAID . . . . . : 178806341
DHCPv6 客户端 DUID . . . . . : 00-01-00-01-27-CE-C5-04-A8-5E-45-C0-E9-BC
DNS 服务器 . . . . . : 202.96.134.133
                          202.96.128.166
TCP/IP 上的 NetBIOS . . . . . : 已启用

PS C:\Windows\system32> ipconfig/all
ipconfig/all : 无法将“ipconfig/all”项识别为 cmdlet、函数、脚本文件或可运行程序的名称。
请确保路径正确，然后再试一次。
所在位置 行:1 字符: 1
```

图 1-2 `ipconfig /all` 的运行结果

1.3 `ipconfig /release`

使用该条指令可以释放（归还）所有接口的租用 `IPv4` 地址。指令结果如图 1-3 所示。

```
PS C:\Windows\system32> ipconfig /release

Windows IP 配置

以太网适配器 VirtualBox Host-Only Network:

    连接特定的 DNS 后缀 . . . . . :
    本地链接 IPv6 地址. . . . . : fe80::28a3:2492:c5e:ad73%11
    IPv4 地址 . . . . . : 192.168.56.1
    子网掩码 . . . . . : 255.255.255.0
    默认网关. . . . . :

以太网适配器 以太网:

    连接特定的 DNS 后缀 . . . . . :
    IPv6 地址 . . . . . : 2001:250:3c00:3509:c078:724a:a230:cff7
    临时 IPv6 地址. . . . . : 2001:250:3c00:3509:357f:b0f9:b1fa:a840
    本地链接 IPv6 地址. . . . . : fe80::c078:724a:a230:cff7%12
    默认网关. . . . . : fe80::3a22:d6ff:fe2b:9aff%12

PS C:\Windows\system32>
```

ipconfig /release指令及运行结果

相对上面的IPv4已经被清除

图 1-3 ipconfig /release 的运行结果

经过测试，在使用了该指令后，校园网环境下的 *drcom* 无法自动重新接口 IPv4 地址，电脑无法再次接入互联网。

1.4 ipconfig /renew

更新所有接口的 IPv4 地址。多数情况下网卡将被重新赋予和以前所赋予的相同的 IP 地址，但租约过期时间会更新。该指令结果如图 1-4 所示。

```
PS C:\Windows\system32> ipconfig /renew

Windows IP 配置

以太网适配器 VirtualBox Host-Only Network:

    连接特定的 DNS 后缀 . . . . . :
    本地链接 IPv6 地址. . . . . : fe80::28a3:2492:c5e:ad73%11
    IPv4 地址 . . . . . : 192.168.56.1
    子网掩码 . . . . . : 255.255.255.0
    默认网关. . . . . :

以太网适配器 以太网:

    连接特定的 DNS 后缀 . . . . . : campus.szu.edu.cn
    IPv6 地址 . . . . . : 2001:250:3c00:3509:c078:724a:a230:cff7
    临时 IPv6 地址. . . . . : 2001:250:3c00:3509:357f:b0f9:b1fa:a840
    本地链接 IPv6 地址. . . . . : fe80::c078:724a:a230:cff7%12
    IPv4 地址 . . . . . : 172.29.39.79
    子网掩码 . . . . . : 255.255.255.0
    默认网关. . . . . : fe80::3a22:d6ff:fe2b:9aff%12
                        172.29.39.1

PS C:\Windows\system32>
```

ipconfig /renew指令及运行结果

IPv4已经更新

图 1-4 ipconfig /renew 的运行结果

经过测试，在使用了该指令后，校园网环境下的 *drcom* 可以再次直接登录成功，无需重启电脑即可连接互联网。

2 ping

ping 是一个测试程序，用于确定本地主机是否能与另一台主机发送或接收数据报。如果 *ping* 运行正确，就可以排除发送与接收方网络层以下的故障。

按缺省设置，运行 *ping* 命令时发送 4 个 *ICMP*（网络控制报文协议）回显请求，每个含 32 字节数据。若正常，应收到 4 个回显应答。关于 *ping* 指令的更多参数设置，如发送报文协议的数量和 *ping* 命令中的数据长度，在 2.6 部分有更详细的说明与运行结果。

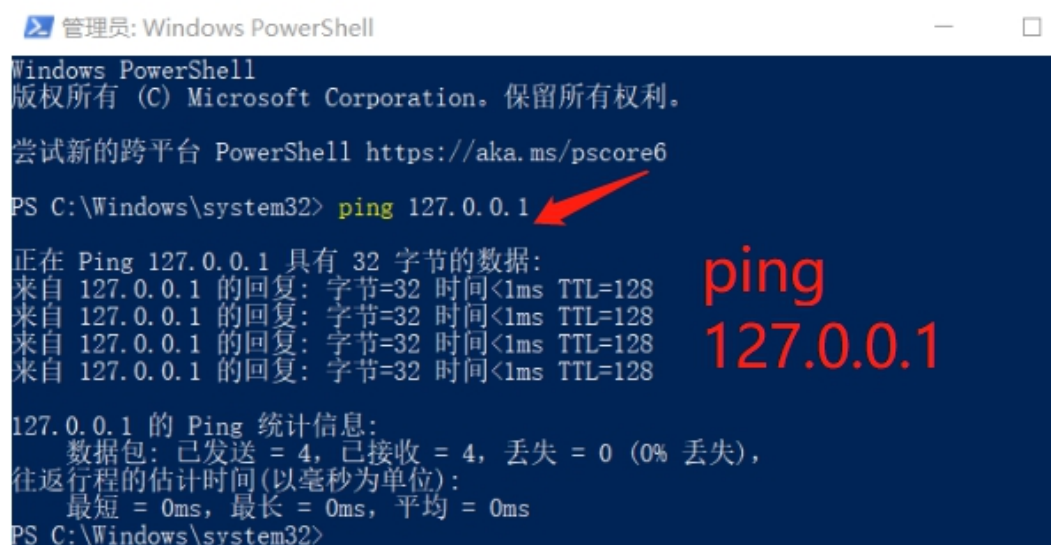
ping 显示发送回显请求到收到回显应答之间的时间间隔，单位为毫秒。

ping 还能显示 *TTL*（*Time To Live*），即生存时间。一般情况下通过 *TTL* 值推算数据包已经通过了多少个路由器：源地点 *TTL* 起始值（就是比返回 *TTL* 略大的一个 2 的乘方数）- 返回时 *TTL* 值。例如，返回 *TTL* 值为 119，那么可以推算数据报离开源地址的 *TTL* 起始值为 128，而源地到目标地要通过 9 个路由器网段（128-119）；如果返回 *TTL* 值为 246，*TTL* 起始值就是 256，源地到目标地要通过 9 个路由器网段。实际上，不同操作系统中 *TTL* 起始值不同，255 是最大值，因为 *TTL* 字段最多 8 位。

关于 *ping*，需要注意其传输方向为本地——被 *ping* 地址，详细的说明会在之后 4.2 与 *tracert* 对比。

2.1 ping 127.0.0.1

这个 *ping* 命令被送到本地计算机的 *IP* 协议层。如果出错则表示 *TCP/IP* 的安装或运行存在某些问题。在本机上该指令的运行结果如图 2-1 所示。可以看到丢失率为 0%，*TCP/IP* 的安装与运行没有问题。



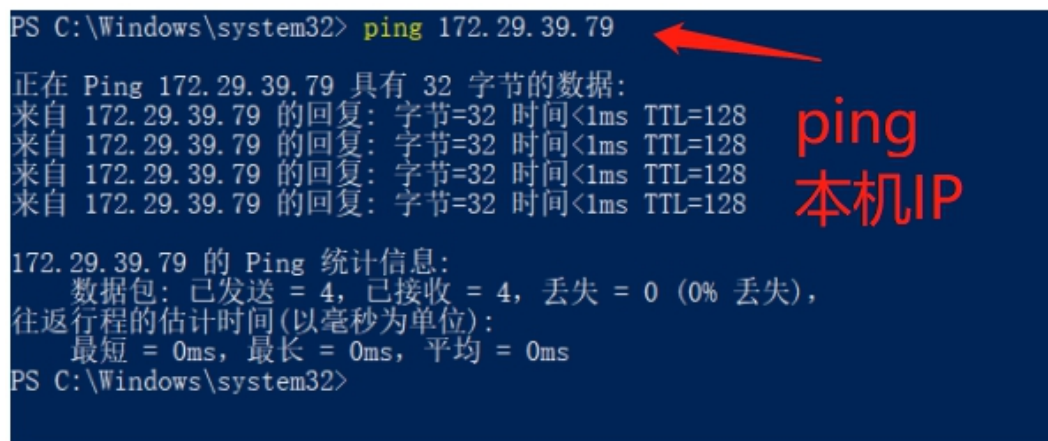
```
管理员: Windows PowerShell
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。
尝试新的跨平台 PowerShell https://aka.ms/pscore6
PS C:\Windows\system32> ping 127.0.0.1
正在 Ping 127.0.0.1 具有 32 字节的数据:
来自 127.0.0.1 的回复: 字节=32 时间<1ms TTL=128
来自 127.0.0.1 的回复: 字节=32 时间<1ms TTL=128
来自 127.0.0.1 的回复: 字节=32 时间<1ms TTL=128
来自 127.0.0.1 的回复: 字节=32 时间<1ms TTL=128
127.0.0.1 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 0ms, 最长 = 0ms, 平均 = 0ms
PS C:\Windows\system32>
```

图 2-1 *ping* 127.0.0.1 的运行结果

2.2 ping 本机 IP (通过 ipconfig 查询)

这个命令被送到本计算机所配置的 IP 地址。如果出错,则表示本地配置或安装存在问题。

由 1.1 的运行结果可知本机 IP 为 172.29.39.79, ping 本机 IP 的运行结果如图 2-2 所示。可以看到丢失率为 0%,本地配置和安装没有问题。



```
PS C:\Windows\system32> ping 172.29.39.79

正在 Ping 172.29.39.79 具有 32 字节的数据:
来自 172.29.39.79 的回复: 字节=32 时间<1ms TTL=128
来自 172.29.39.79 的回复: 字节=32 时间<1ms TTL=128
来自 172.29.39.79 的回复: 字节=32 时间<1ms TTL=128
来自 172.29.39.79 的回复: 字节=32 时间<1ms TTL=128

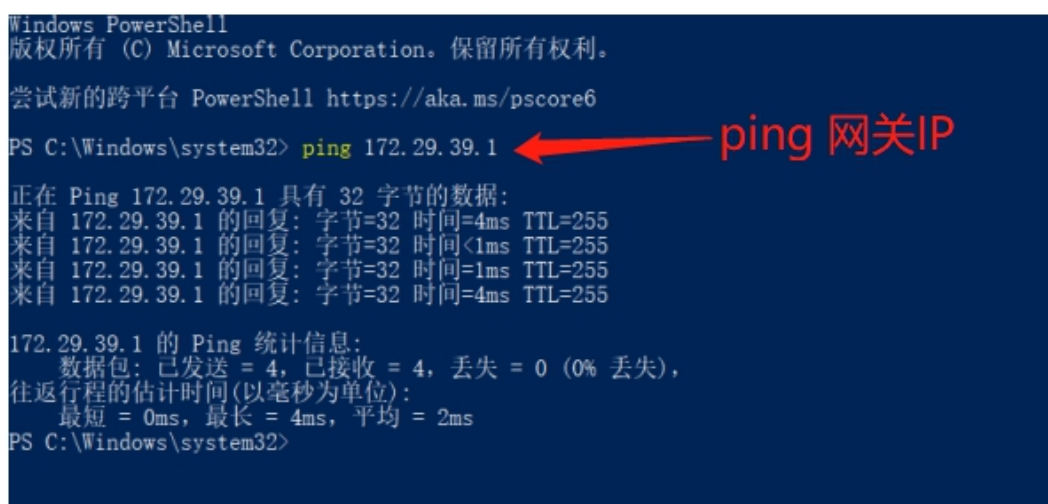
172.29.39.79 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 0ms, 最长 = 0ms, 平均 = 0ms
PS C:\Windows\system32>
```

图 2-2 ping 本机 IP 的运行结果

2.3 ping 网关 IP

ping 网关 IP 如果能够应答正确,表示局域网中的网关路由器正在运行并能够做出应答。

由 1.1 的运行结果可知网关 IP 为 172.29.39.1, ping 网关 IP 的运行结果如图 2-3 所示。可以看到丢失率为 0%,网关路由器正在运行,没有问题。



```
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。

尝试新的跨平台 PowerShell https://aka.ms/pscore6

PS C:\Windows\system32> ping 172.29.39.1

正在 Ping 172.29.39.1 具有 32 字节的数据:
来自 172.29.39.1 的回复: 字节=32 时间=4ms TTL=255
来自 172.29.39.1 的回复: 字节=32 时间<1ms TTL=255
来自 172.29.39.1 的回复: 字节=32 时间=1ms TTL=255
来自 172.29.39.1 的回复: 字节=32 时间=4ms TTL=255

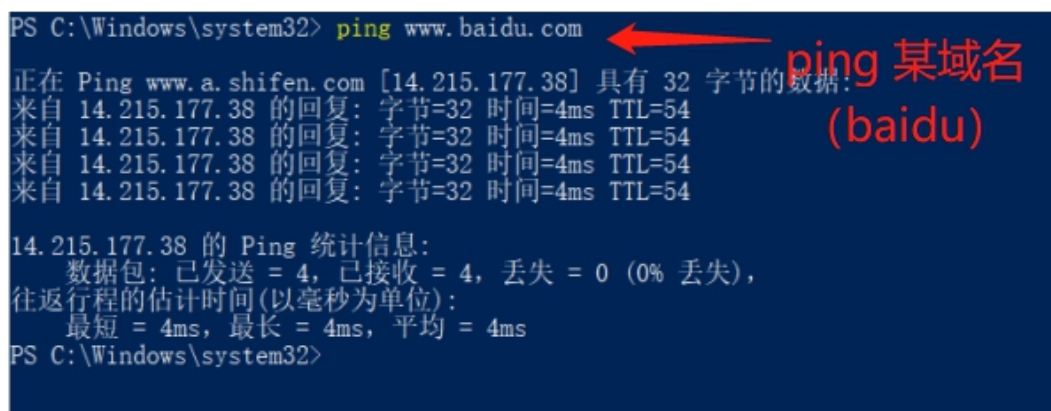
172.29.39.1 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 0ms, 最长 = 4ms, 平均 = 2ms
PS C:\Windows\system32>
```

图 2-3 ping 网关 IP 的运行结果

2.4 ping 某域名 (www.baidu.com)

对某个域名执行 ping 命令,本地计算机必须先通过 DNS 服务器将域名转

换成 IP 地址。如果出现故障,则表示 DNS 服务器的 IP 地址配置不正确或 DNS 服务器有故障。这里以 baidu 为例, ping www.baidu.com 的运行结果如图 2-4 所示。可以看到能够成功 ping 通, 连接没有问题。



```
PS C:\Windows\system32> ping www.baidu.com
正在 Ping www.a.shifen.com [14.215.177.38] 具有 32 字节的数据:
来自 14.215.177.38 的回复: 字节=32 时间=4ms TTL=54
来自 14.215.177.38 的回复: 字节=32 时间=4ms TTL=54
来自 14.215.177.38 的回复: 字节=32 时间=4ms TTL=54
来自 14.215.177.38 的回复: 字节=32 时间=4ms TTL=54

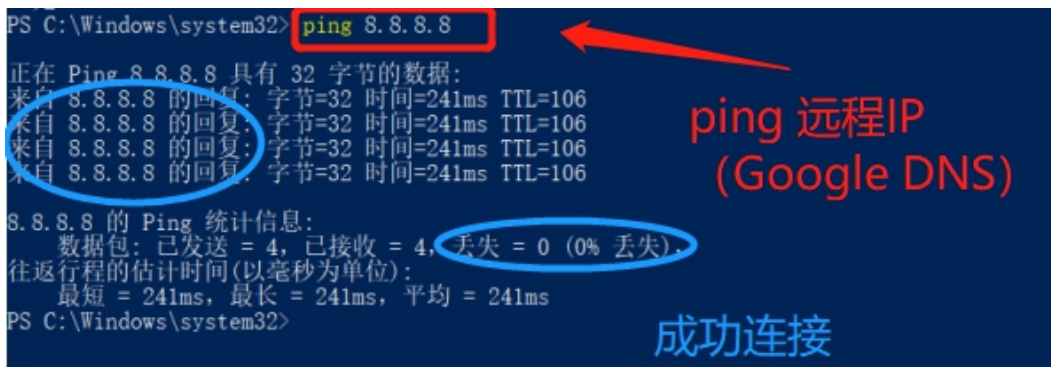
14.215.177.38 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 4ms, 最长 = 4ms, 平均 = 4ms
PS C:\Windows\system32>
```

图 2-4 ping www.baidu.com (某域名) 的运行结果

2.5 ping 远程 IP

在 ping 远程 IP 时,如果能够收到 4 个应答,则表示成功使用了缺省网关。对于拨号上网用户则表示能够成功的访问 Internet(但不排除 ISP 的 DNS 会有问题)。

在本次实验中,远程 IP 使用 Google 提供的免费 DNS 服务器: 8.8.8.8。ping 远程 IP 的指令为: ping 8.8.8.8, 运行结果如图 2-5 所示, 可以看到丢失率为 0%, 远程 IP 可以 ping 通。



```
PS C:\Windows\system32> ping 8.8.8.8
正在 Ping 8.8.8.8 具有 32 字节的数据:
来自 8.8.8.8 的回复: 字节=32 时间=241ms TTL=106
来自 8.8.8.8 的回复: 字节=32 时间=241ms TTL=106
来自 8.8.8.8 的回复: 字节=32 时间=241ms TTL=106
来自 8.8.8.8 的回复: 字节=32 时间=241ms TTL=106

8.8.8.8 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 241ms, 最长 = 241ms, 平均 = 241ms
PS C:\Windows\system32>
```

图 2-5 ping 8.8.8.8 (远程 IP) 的运行结果

2.6 ping -常用参数选项

(1) ping IP -t

该指令连续对 IP 地址执行 ping 命令, 直到被用户以 Ctrl+C 中断。IP 地址以 Google 提供的免费 DNS 服务器: 8.8.8.8 为例, 运行结果如图 2-6-1 所示。可以看到, -t 可以一直执行 ping 命令直到被用户中断。


```
PS C:\Windows\system32> ping 8.8.8.8 -t ping IP -t
正在 Ping 8.8.8.8 具有 32 字节的数据:
来自 8.8.8.8 的回复: 字节=32 时间=241ms TTL=106
来自 8.8.8.8 的回复: 字节=32 时间=241ms TTL=106
来自 8.8.8.8 的回复: 字节=32 时间=241ms TTL=106
来自 8.8.8.8 的回复: 字节=32 时间=242ms TTL=106
来自 8.8.8.8 的回复: 字节=32 时间=242ms TTL=106
来自 8.8.8.8 的回复: 字节=32 时间=241ms TTL=106
来自 8.8.8.8 的回复: 字节=32 时间=242ms TTL=106
来自 8.8.8.8 的回复: 字节=32 时间=241ms TTL=106
来自 8.8.8.8 的回复: 字节=32 时间=241ms TTL=106
来自 8.8.8.8 的回复: 字节=32 时间=241ms TTL=106
来自 8.8.8.8 的回复: 字节=32 时间=242ms TTL=106
来自 8.8.8.8 的回复: 字节=32 时间=242ms TTL=106
来自 8.8.8.8 的回复: 字节=32 时间=241ms TTL=106
8.8.8.8 的 Ping 统计信息:
    数据包: 已发送 = 13, 已接收 = 13, 丢失 = 0 (0% 丢失),
往返行程的估计时间(以毫秒为单位):
    最短 = 241ms, 最长 = 242ms, 平均 = 241ms
Control-C
PS C:\Windows\system32> Ctrl+C终止
```

图 2-6-1 `ping 8.8.8.8 -t` 的运行结果

(2) `ping IP -l size`

该指令可以指定 `ping` 命令中的数据长度为 `size` 字节，缺省为 32 字节。实验以 8.8.8.8 为远程 IP 地址，64 字节为例，输入指令：`ping 8.8.8.8 -l 64`，得到结果如图 2-6-2 所示。

```
PS C:\Windows\system32> ping 8.8.8.8 -l 64 ping IP -l size
正在 Ping 8.8.8.8 具有 64 字节的数据:
来自 8.8.8.8 的回复: 字节=64 时间=241ms TTL=106
来自 8.8.8.8 的回复: 字节=64 时间=241ms TTL=106
来自 8.8.8.8 的回复: 字节=64 时间=242ms TTL=106
来自 8.8.8.8 的回复: 字节=64 时间=243ms TTL=106
8.8.8.8 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
往返行程的估计时间(以毫秒为单位):
    最短 = 241ms, 最长 = 243ms, 平均 = 241ms
PS C:\Windows\system32>
```

图 2-6-2 `ping 8.8.8.8 -l 64` 的运行结果

(3) `ping IP -n count`

该指令可以指定 `ping` 命令中执行 `count` 次数的 `ping` 命令，缺省时为 4 次。实验以 8.8.8.8 为远程 IP 地址，8 次为例，输入指令：`ping 8.8.8.8 -n 8`，得到

结果如图 2-6-3 所示。

```
PS C:\Windows\system32> ping 8.8.8.8 -n 8
正在 Ping 8.8.8.8 具有 32 字节的数据:
来自 8.8.8.8 的回复: 字节=32 时间=259ms TTL=106
请求超时。
来自 8.8.8.8 的回复: 字节=32 时间=241ms TTL=106
来自 8.8.8.8 的回复: 字节=32 时间=245ms TTL=106
来自 8.8.8.8 的回复: 字节=32 时间=244ms TTL=106
来自 8.8.8.8 的回复: 字节=32 时间=252ms TTL=106
来自 8.8.8.8 的回复: 字节=32 时间=243ms TTL=106
来自 8.8.8.8 的回复: 字节=32 时间=243ms TTL=106

8.8.8.8 的 Ping 统计信息:
    数据包: 已发送 = 8, 已接收 = 7, 丢失 = 1 (12% 丢失),
往返行程的估计时间(以毫秒为单位):
    最短 = 241ms, 最长 = 259ms, 平均 = 246ms
PS C:\Windows\system32>
```

ping IP -n count

图 2-6-3 ping 8.8.8.8 -n 8 的运行结果

2.7 ping 参数用法查询

ping 命令的参数用法查询指令为: ping。输入该指令后, 可以得到关于 ping 的选项与详细用法, 如图 2-7 所示。(经修正, 不是 ping -h, ping 即可)

```
PS C:\Windows\system32> ping -h
选项 -h 不正确。
用法: ping [-t] [-a] [-n count] [-l size] [-f] [-i TTL] [-v TOS]
        [-r count] [-s count] [[-j host-list] | [-k host-list]]
        [-w timeout] [-R] [-S srcaddr] [-c compartment] [-p]
        [-4] [-6] target_name

选项:
    -t          Ping 指定的主机, 直到停止。
                若要查看统计信息并继续操作, 请键入 Ctrl+Break;
                若要停止, 请键入 Ctrl+C。
    -a          将地址解析为主机名。
    -n count    要发送的回显请求数。
    -l size     发送缓冲区大小。
    -f          在数据包中设置“不分段”标记(仅适用于 IPv4)。
    -i TTL      生存时间。
    -v TOS      服务类型(仅适用于 IPv4。该设置已被弃用,
                对 IP 标头中的服务类型字段没有任何影响)。
    -r count    记录计数跃点的路由(仅适用于 IPv4)。
    -s count    计数跃点的时间戳(仅适用于 IPv4)。
    -j host-list 与主机列表一起使用的松散源路由(仅适用于 IPv4)。
    -k host-list 与主机列表一起使用的严格源路由(仅适用于 IPv4)。
    -w timeout  等待每次回复的超时时间(毫秒)。
    -R          同样使用路由标头测试反向路由(仅适用于 IPv6)。
                根据 RFC 5095, 已弃用此路由标头。
                如果使用此标头, 某些系统可能丢弃回显请求。
    -S srcaddr  要使用的源地址。
    -c compartment 路由隔离舱标识符。
    -p          Ping Hyper-V 网络虚拟化提供程序地址。
    -4          强制使用 IPv4。
    -6          强制使用 IPv6。
```

ping -h

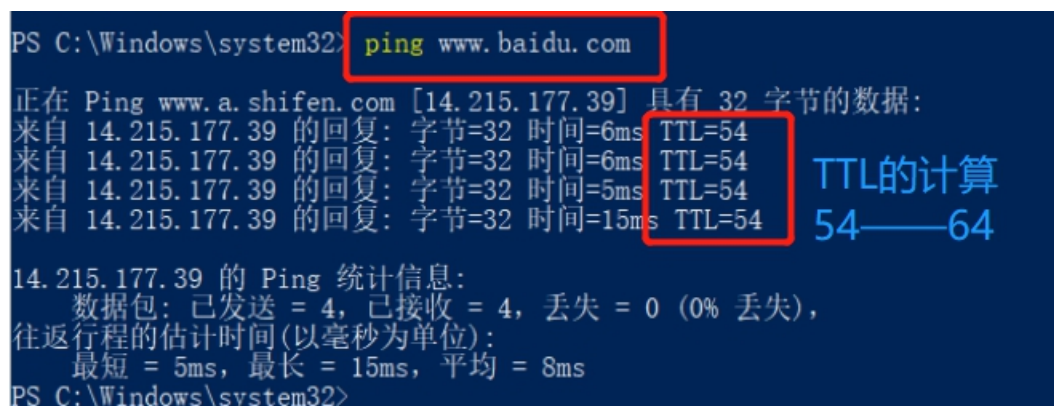
经修正, 直接 ping 即可

图 2-7 ping 的运行结果

2.8 利用 *TTL* 计算源节点与目的节点之间路由器数量

ping 能显示 *TTL* (*Time To Live*)，即生存时间。一般情况下通过 *TTL* 值推算数据包已经通过了多少个路由器：源地点 *TTL* 起始值（就是比返回 *TTL* 略大的一个 2 的乘方数）-返回时 *TTL* 值。例如，返回 *TTL* 值为 119，那么可以推算数据报离开源地址的 *TTL* 起始值为 128，而源地到目标地要通过 9 个路由器网段（128-119）；如果返回 *TTL* 值为 246，*TTL* 起始值就是 256，源地到目标地要通过 9 个路由器网段。实际上，不同操作系统中 *TTL* 起始值不同，255 是最大值，因为 *TTL* 字段最多 8 位。

以 *ping www.baidu.com* 指令为例，*baidu* 的 *IP* 地址实际为 14.215.177.39，由图 2-8 可以看到返回的 *TTL*=54，由此计算经过了 10 个路由器（64-54），最终到达接收方。注意传输方向为：本机←——14.215.177.39。



```
PS C:\Windows\system32> ping www.baidu.com

正在 Ping www.a.shifen.com [14.215.177.39] 具有 32 字节的数据:
来自 14.215.177.39 的回复: 字节=32 时间=6ms TTL=54
来自 14.215.177.39 的回复: 字节=32 时间=6ms TTL=54
来自 14.215.177.39 的回复: 字节=32 时间=5ms TTL=54
来自 14.215.177.39 的回复: 字节=32 时间=15ms TTL=54

14.215.177.39 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 5ms, 最长 = 15ms, 平均 = 8ms
PS C:\Windows\system32>
```

TTL的计算
54——64

图 2-8 *ping www.baidu.com*，根据 *TTL* 计算源节点与目的节点之间路由器数量

3 netstat

netstat 用于显示 *IP*、*TCP*、*UDP* 和 *ICMP* 协议的统计信息，用于检验本机各端口网络连接情况。

3.1 netstat -s

该指令用于显示每个协议的统计信息。默认情况下该指令显示 *IP*、*IPv6*、*ICMP*、*ICMPv6*、*TCP*、*TCPv6*、*UDP* 和 *UDPv6* 的统计信息。运行结果如图 3-1 所示。（由于该指令回显内容很多，截图需要分开截取）

PS C:\Windows\system32> netstat -s

netstat -s指令

IPv4 统计信息

IPv4信息

接收的数据包	= 2889801
接收的标头错误	= 0
接收的地址错误	= 5001
转发的数据报	= 0
接收的未知协议	= 0
丢弃的接收数据包	= 71762
传送的接收数据包	= 2870162
输出请求	= 2178007
路由丢弃	= 0
丢弃的输出数据包	= 6
输出数据包无路由	= 121
需要重新组合	= 2
重新组合成功	= 1
重新组合失败	= 0
数据报分段成功	= 0
数据报分段失败	= 0
分段已创建	= 0

IPv6 统计信息

IPv6信息

接收的数据包	= 529326
接收的标头错误	= 0
接收的地址错误	= 100
转发的数据报	= 0
接收的未知协议	= 0
丢弃的接收数据包	= 19087
传送的接收数据包	= 541328
输出请求	= 344238
路由丢弃	= 0
丢弃的输出数据包	= 5
输出数据包无路由	= 10
需要重新组合	= 0
重新组合成功	= 0
重新组合失败	= 0
数据报分段成功	= 0
数据报分段失败	= 0
分段已创建	= 0

ICMPv4 统计信息

		已接收	已发送
消息		8528	430
错误		10	0
目标不可达	7732	325	
超时	548	0	
参数问题	0	0	

源抑制	0	0	
重定向		149	0
回显回复		68	8
回显		21	97
时间戳		0	0
时间戳回复	0		0
地址掩码	0		0
地址掩码回复	0		0
路由器请求	0	0	
路由器播发	0	0	


```
ICMPv6 统计信息

消息                                已接收  已发送
错误                                774    415
目标不可达                        152    0
数据包太大                        0      0
超时                              0      0
参数问题                          0      0
回显                              0      0
回显回复                          0      0
MLD 查询                          0      0
MLD 报告                          0      0
MLD 已完成                        0      0
路由器请求                        0      27
路由器播发                        149    0
邻居请求                          143    167
邻居播发                          330    167
重定向                            0      0
路由器重新编号                    0      0

IPv4 的 TCP 统计信息

主动开放                          = 9844
被动开放                          = 80
失败的连接尝试                    = 2052
重置连接                          = 619
当前连接                          = 29
接收的分段                        = 1954088
发送的分段                        = 1221002
重新传输的分段                    = 0

IPv6 的 TCP 统计信息

主动开放                          = 2153
被动开放                          = 82
失败的连接尝试                    = 288
重置连接                          = 54
当前连接                          = 14
接收的分段                        = 207138
发送的分段                        = 108275
重新传输的分段                    = 0

IPv4 的 UDP 统计信息

接收的数据报                      = 1075402
无端口                            = 36417
接收错误                          = 26755
发送的数据报                      = 800976

IPv6 的 UDP 统计信息

接收的数据报                      = 500370
无端口                            = 764
接收错误                          = 18172
发送的数据报                      = 218109
PS C:\Windows\system32>
```

图 3-1 netstat -s 的运行结果

3.2 netstat -e

该指令用于显示以太网统计信息；运行结果如图 3-2 所示。

```
PS C:\Windows\system32> netstat -e
接口统计

接收的          发送的
字节            2749096872      3272096810
单播数据包      12825525        9735295
非单播数据包    966165          48645
丢弃            0               0
错误            0               0
未知协议        0               0
PS C:\Windows\system32>
```

图 3-2 netstat -e 的运行结果

3.3 netstat -r

该指令用于显示每个协议的统计信息。默认情况下，显示 IP、IPv6、ICMP、ICMPv6、TCP、TCPv6、UDP 和 UDPv6 的统计信息。指令的运行结果如图 3-3 所示。

```
PS C:\Windows\system32> netstat -r
===== netstat -r 显示每个协议信息 =====
接口列表
11...0a 00 27 00 00 0b .....VirtualBox Host-Only Ethernet Adapter
6...60 f2 62 74 6b 1b .....Intel(R) Wireless-AC 9462
2...60 f2 62 74 6b 1c .....Microsoft Wi-Fi Direct Virtual Adapter
3...62 f2 62 74 6b 1b .....Microsoft Wi-Fi Direct Virtual Adapter #2
12...a8 5e 45 c0 e9 bc .....Realtek PCIe GbE Family Controller
1.....Software Loopback Interface 1
=====

IPv4 路由表
=====
活动路由:
网络目标      网络掩码      网关      接口      跃点数
0.0.0.0        0.0.0.0        172.29.39.1  172.29.39.79  35
127.0.0.0      255.0.0.0      在链路上    127.0.0.1     331
127.0.0.1      255.255.255.255 在链路上    127.0.0.1     331
127.255.255.255 255.255.255.255 在链路上    127.0.0.1     331
172.29.39.0    255.255.255.0   在链路上    172.29.39.79  291
172.29.39.79   255.255.255.255 在链路上    172.29.39.79  291
172.29.39.255  255.255.255.255 在链路上    172.29.39.79  291
172.30.255.2   255.255.255.255 172.29.39.1  172.29.39.79  36
192.168.56.0   255.255.255.0   在链路上    192.168.56.1  281
192.168.56.1   255.255.255.255 在链路上    192.168.56.1  281
192.168.56.255 255.255.255.255 在链路上    192.168.56.1  281
224.0.0.0      240.0.0.0       在链路上    127.0.0.1     331
224.0.0.0      240.0.0.0       在链路上    192.168.56.1  281
224.0.0.0      240.0.0.0       在链路上    172.29.39.79  291
255.255.255.255 255.255.255.255 在链路上    127.0.0.1     331
255.255.255.255 255.255.255.255 在链路上    192.168.56.1  281
255.255.255.255 255.255.255.255 在链路上    172.29.39.79  291
=====
永久路由:
无
```

```
IPv6 路由表
=====
活动路由:
接口跃点数网络目标          网关
12 291 ::/0                  fe80::3a22:d6ff:fe2b:9aff
1 331 ::1/128                在链路上
12 291 2001:250:3c00:3509::/64 在链路上
12 291 2001:250:3c00:3509:571:e64e:f252:e982/128
在链路上
12 291 2001:250:3c00:3509:c078:724a:a230:cff7/128
在链路上
11 281 fe80::/64              在链路上
12 291 fe80::/64              在链路上
11 281 fe80::28a3:2492:c5e:ad73/128
在链路上
12 291 fe80::c078:724a:a230:cff7/128
在链路上
1 331 ff00::/8                在链路上
11 281 ff00::/8                在链路上
12 291 ff00::/8                在链路上
=====
永久路由:
无
PS C:\Windows\system32>
```

图 3-3 netstat -r 的运行结果

3.4 netstat -a

该指令显示所有连接和侦听端口。所显示的状态有：已建立（ESTABLISHED）、正在监听（LISTENING）、TCP 握手（SYN_SENT）等。指令运行结果如图 3-4 所示。

```
PS C:\Windows\system32> netstat -a
活动连接
协议 本地地址          外部地址          状态
TCP 0.0.0.0:135        DESKTOP-BKQJ7B9:0 LISTENING
TCP 0.0.0.0:445        DESKTOP-BKQJ7B9:0 LISTENING
TCP 0.0.0.0:5040       DESKTOP-BKQJ7B9:0 LISTENING
TCP 0.0.0.0:5357       DESKTOP-BKQJ7B9:0 LISTENING
TCP 0.0.0.0:7680       DESKTOP-BKQJ7B9:0 LISTENING
TCP 0.0.0.0:8680       DESKTOP-BKQJ7B9:0 LISTENING
TCP 0.0.0.0:49664      DESKTOP-BKQJ7B9:0 LISTENING
TCP 0.0.0.0:49665      DESKTOP-BKQJ7B9:0 LISTENING
TCP 0.0.0.0:49666      DESKTOP-BKQJ7B9:0 LISTENING
TCP 0.0.0.0:49667      DESKTOP-BKQJ7B9:0 LISTENING
TCP 0.0.0.0:49668      DESKTOP-BKQJ7B9:0 LISTENING
TCP 0.0.0.0:49677      DESKTOP-BKQJ7B9:0 LISTENING
TCP 0.0.0.0:53125      DESKTOP-BKQJ7B9:0 LISTENING
TCP 127.0.0.1:4301      DESKTOP-BKQJ7B9:0 LISTENING
TCP 127.0.0.1:4709      DESKTOP-BKQJ7B9:0 LISTENING
TCP 127.0.0.1:5939      DESKTOP-BKQJ7B9:0 LISTENING
TCP 127.0.0.1:50000     DESKTOP-BKQJ7B9:0 LISTENING
TCP 127.0.0.1:57936     DESKTOP-BKQJ7B9:0 LISTENING
TCP 172.29.39.79:139    DESKTOP-BKQJ7B9:0 LISTENING
TCP 172.29.39.79:55688  120.92.103.226:10002 ESTABLISHED
TCP 172.29.39.79:56155  52.139.250.253:https ESTABLISHED
TCP 172.29.39.79:56170  58.60.9.104:https    ESTABLISHED
TCP 172.29.39.79:57820  113.96.202.105:8080  ESTABLISHED
TCP 172.29.39.79:57906  14.18.180.113:https  CLOSE_WAIT
TCP 172.29.39.79:57918  14.215.158.24:https  CLOSE_WAIT
TCP 172.29.39.79:57939  14.18.180.113:https  CLOSE_WAIT
TCP 172.29.39.79:57941  14.18.180.113:https  CLOSE_WAIT
TCP 172.29.39.79:57943  14.18.180.113:https  CLOSE_WAIT
TCP 172.29.39.79:57971  183.3.225.71:https   CLOSE_WAIT
TCP 172.29.39.79:57972  14.215.158.24:https  CLOSE_WAIT
TCP 172.29.39.79:58003  14.215.85.107:https  CLOSE_WAIT
TCP 172.29.39.79:58013  121.9.212.144:http   CLOSE_WAIT
TCP 172.29.39.79:58014  121.9.212.144:http   CLOSE_WAIT
TCP 172.29.39.79:58016  121.9.212.144:http   CLOSE_WAIT
TCP 172.29.39.79:58017  121.9.212.144:http   CLOSE_WAIT
TCP 172.29.39.79:58112  183.3.226.50:https   CLOSE_WAIT
TCP 172.29.39.79:58541  113.96.237.36:https  CLOSE_WAIT
TCP 172.29.39.79:58807  13.107.5.88:https    ESTABLISHED
TCP 172.29.39.79:58911  59.37.97.23:https    CLOSE_WAIT
TCP 172.29.39.79:58983  223.252.199.69:6003  ESTABLISHED
```

netstat -a显示所有连接与侦听端口

```

TCP 172.29.39.79:58983 223.252.199.69:6003 ESTABLISHED
TCP 172.29.39.79:59473 121.9.212.144:http CLOSE_WAIT
TCP 172.29.39.79:59680 59.37.97.23:https CLOSE_WAIT
TCP 172.29.39.79:59782 14.18.180.162:https CLOSE_WAIT
TCP 172.29.39.79:59801 110.43.195.11:https CLOSE_WAIT
TCP 172.29.39.79:59849 180.163.151.162:https TIME_WAIT
TCP 172.29.39.79:59887 20.44.232.74:https ESTABLISHED
TCP 172.29.39.79:59920 14.215.177.38:http ESTABLISHED
TCP 172.29.39.79:59922 101.91.63.187:https TIME_WAIT
TCP 172.29.39.79:59923 106.124.127.21:http TIME_WAIT
TCP 172.29.39.79:59924 106.124.127.22:http TIME_WAIT
TCP 172.29.39.79:59926 123.150.208.86:8080 TIME_WAIT
TCP 172.29.39.79:59927 106.124.127.23:http TIME_WAIT
TCP 172.29.39.79:59928 14.18.161.24:http TIME_WAIT
TCP 172.29.39.79:59929 222.217.93.192:http TIME_WAIT
TCP 172.29.39.79:59930 124.232.141.139:https TIME_WAIT
TCP 172.29.39.79:59931 161:http CLOSE_WAIT
TCP 172.29.39.79:59947 14.215.177.38:https FIN_WAIT_1
TCP 172.29.39.79:59948 14.18.161.24:http ESTABLISHED
TCP 172.29.39.79:59949 14.215.177.38:https FIN_WAIT_1
TCP 192.168.56.1:139 DESKTOP-BKQJ7B9:0 LISTENING
TCP [::]:135 DESKTOP-BKQJ7B9:0 LISTENING
TCP [::]:445 DESKTOP-BKQJ7B9:0 LISTENING
TCP [::]:5357 DESKTOP-BKQJ7B9:0 LISTENING
TCP [::]:7680 DESKTOP-BKQJ7B9:0 LISTENING
TCP [::]:49664 DESKTOP-BKQJ7B9:0 LISTENING
TCP [::]:49665 DESKTOP-BKQJ7B9:0 LISTENING
TCP [::]:49666 DESKTOP-BKQJ7B9:0 LISTENING
TCP [::]:49667 DESKTOP-BKQJ7B9:0 LISTENING
TCP [::]:49668 DESKTOP-BKQJ7B9:0 LISTENING
TCP [::]:49677 DESKTOP-BKQJ7B9:0 LISTENING
TCP [::]:53125 DESKTOP-BKQJ7B9:0 LISTENING
TCP [::]:49671 DESKTOP-BKQJ7B9:0 LISTENING
TCP [2001:250:3c00:3509:571:e64e:f252:e982]:58020 [240e:ff:d188:700:3::3fd]:http CLOSE_WAIT
TCP [2001:250:3c00:3509:571:e64e:f252:e982]:59240 [2407:b380:200:1000:103:72:47:241]:http CLOSE_WAIT
TCP [2001:250:3c00:3509:571:e64e:f252:e982]:59474 [240e:ff:f000:300:3::3fd]:http CLOSE_WAIT
TCP [2001:250:3c00:3509:571:e64e:f252:e982]:59933 [2407:b380:200:1000:103:72:47:241]:http CLOSE_WAIT
TCP [2001:250:3c00:3509:571:e64e:f252:e982]:59995 [240e:83:205:8:0:ff:b068:3ce]:https ESTABLISHED
TCP [2001:250:3c00:3509:571:e64e:f252:e982]:59996 [240e:83:205:8:0:ff:b068:3ce]:https ESTABLISHED
UDP 0.0.0.0:500 *: *
UDP 0.0.0.0:3702 *: *
UDP 0.0.0.0:3702 *: *
UDP 0.0.0.0:4012 *: *
UDP 0.0.0.0:4500 *: *
UDP 0.0.0.0:5050 *: *
UDP 0.0.0.0:5353 *: *
UDP 0.0.0.0:5353 *: *
UDP 0.0.0.0:5353 *: *
UDP 0.0.0.0:5353 *: *
UDP 0.0.0.0:5353 *: *
UDP 0.0.0.0:5353 *: *
UDP 0.0.0.0:5353 *: *
UDP 0.0.0.0:5353 *: *
UDP 0.0.0.0:5353 *: *
UDP 0.0.0.0:5353 *: *
UDP 0.0.0.0:5353 *: *
UDP 0.0.0.0:5353 *: *

```

图 3-4 netstat -a 的运行结果

3.5 netstat -n

该指令可以显示所有的活动连接，并且以数字形式显示地址和端口号。运行结果如图 3-5 所示。

```

PS C:\Windows\system32> netstat -n
活动连接
 协议 本地地址           外部地址           状态           时间等待
TCP    172.29.39.79:7680   172.29.44.69:65421 ESTABLISHED    0:00:00
TCP    172.29.39.79:55688 120.92.103.226:10002 ESTABLISHED    0:00:00
TCP    172.29.39.79:56155 52.139.250.253:443 ESTABLISHED    0:00:00
TCP    172.29.39.79:56170 58.60.9.104:443 ESTABLISHED    0:00:00
TCP    172.29.39.79:57820 113.96.202.105:8080 ESTABLISHED    0:00:00
TCP    172.29.39.79:57906 14.18.180.113:443 CLOSE_WAIT    0:00:00
TCP    172.29.39.79:57918 14.215.158.24:443 CLOSE_WAIT    0:00:00
TCP    172.29.39.79:57939 14.18.180.113:443 CLOSE_WAIT    0:00:00
TCP    172.29.39.79:57941 14.18.180.113:443 CLOSE_WAIT    0:00:00
TCP    172.29.39.79:57943 14.18.180.113:443 CLOSE_WAIT    0:00:00
TCP    172.29.39.79:57971 183.3.225.71:443 CLOSE_WAIT    0:00:00
TCP    172.29.39.79:57972 14.215.158.24:443 CLOSE_WAIT    0:00:00
TCP    172.29.39.79:58003 14.215.85.107:443 CLOSE_WAIT    0:00:00
TCP    172.29.39.79:58013 121.9.212.144:80 CLOSE_WAIT    0:00:00
TCP    172.29.39.79:58014 121.9.212.144:80 CLOSE_WAIT    0:00:00
TCP    172.29.39.79:58016 121.9.212.144:80 CLOSE_WAIT    0:00:00
TCP    172.29.39.79:58017 121.9.212.144:80 CLOSE_WAIT    0:00:00
TCP    172.29.39.79:58112 183.3.226.50:443 CLOSE_WAIT    0:00:00
TCP    172.29.39.79:58541 113.96.237.36:443 CLOSE_WAIT    0:00:00
TCP    172.29.39.79:58807 13.107.5.88:443 ESTABLISHED    0:00:00
TCP    172.29.39.79:58983 223.252.199.69:6003 ESTABLISHED    0:00:00

```

netstat -n显示所有活动连接


```
TCP 172.29.39.79:59473 121.9.212.144:80 CLOSE_WAIT
TCP 172.29.39.79:59680 59.37.97.23:443 CLOSE_WAIT
TCP 172.29.39.79:59782 14.18.180.162:443 CLOSE_WAIT
TCP 172.29.39.79:59948 14.18.161.24:80 CLOSE_WAIT
TCP 172.29.39.79:60063 59.37.97.23:443 CLOSE_WAIT
TCP 172.29.39.79:60146 110.43.195.11:443 CLOSE_WAIT
TCP 172.29.39.79:60256 52.184.216.226:443 ESTABLISHED
TCP 172.29.39.79:60257 14.215.177.38:443 FIN_WAIT_1
TCP 172.29.39.79:60260 113.105.172.49:443 ESTABLISHED
TCP 172.29.39.79:60261 14.215.177.39:80 ESTABLISHED
TCP 172.29.39.79:60262 14.215.177.38:443 FIN_WAIT_1
TCP 172.29.39.79:60263 110.43.89.215:80 TIME_WAIT
TCP 172.29.39.79:60264 110.43.89.215:80 TIME_WAIT
TCP [2001:250:3c00:3509:571:e64e:f252:e982]:58020 [240e:ff:d188:700:3::3fd]:80 CLOSE_WAIT
TCP [2001:250:3c00:3509:571:e64e:f252:e982]:59240 [2407:b380:200:1000:103:72:47:241]:80 CLOSE_WAIT
TCP [2001:250:3c00:3509:571:e64e:f252:e982]:59474 [240e:ff:f000:300:3::3fd]:80 CLOSE_WAIT
TCP [2001:250:3c00:3509:571:e64e:f252:e982]:60081 [2407:b380:200:1000:103:72:47:241]:80 CLOSE_WAIT
TCP [2001:250:3c00:3509:571:e64e:f252:e982]:60203 [240e:ff:d190:601::7160:b223]:443 TIME_WAIT
TCP [2001:250:3c00:3509:571:e64e:f252:e982]:60251 [240e:83:205:8:0:ff:b068:3ce]:443 CLOSE_WAIT
TCP [2001:250:3c00:3509:571:e64e:f252:e982]:60252 [240e:83:205:8:0:ff:b068:3ce]:443 ESTABLISHED
PS C:\Windows\system32>
```

图 3-5 netstat -n 的运行结果

4 tracert

4.1 tracert 使用方法

tracert 命令可以用来跟踪数据报使用的路由（路径），并列出所经过的每个路由器上所花费的时间。因此，*tracert* 一般用来检测故障的位置。该指令的用法只需在 *tracert* 后面跟上一个 IP 地址或主机名，如：*tracert www.baidu.com*。该指令的运行结果如图 4-1 所示。

```
PS C:\Windows\system32> tracert www.baidu.com tracert IP
通过最多 30 个跃点跟踪
到 www.a.shifen.com [14.215.177.38] 的路由:

 1      *          *          *          请求超时。
 2      <1 毫秒    <1 毫秒    7 ms     192.168.255.179
 3      *          *          *          请求超时。
 4      2 ms       1 ms       1 ms     183.59.55.37
 5      1 ms       1 ms       1 ms     183.56.67.109
 6      4 ms       4 ms       6 ms     113.106.34.249
 7      12 ms      *          11 ms     113.96.5.42
 8      7 ms       7 ms       5 ms     86.96.135.219.broad.fs.gd.dynamic.163data
.com.cn [219.135.96.86]
 9      5 ms       5 ms       5 ms     14.29.121.190
10      *          *          *          请求超时。
11      *          *          *          请求超时。
12      4 ms       4 ms       4 ms     14.215.177.38

跟踪完成。
```

图 4-1 tracert www.baidu.com 的运行结果

4.2 对比 ping 和 tracert

对于相同的网站，可以看到 *ping* 的传输方向为本机<——14.215.177.39，*tracert* 的传输方向为本机——>14.215.177.39，由图 2-4 可知，*ping* *www.baidu.com* 的 TTL 为 54，经过了 10 个路由器；此处 *tracert* 追踪经过了 11 个路由器，说明了两个方向经过的路由很可能不同。

5 ARP（地址转换协议）

显示和修改地址解析协议（ARP）使用的“IP 到物理”地址转换表。

ARP 协议用于确定对应 IP 地址的网卡物理地址。

5.1 arp -a

通过询问当前协议数据，显示当前 ARP 项。如果不止一个网络接口使用 ARP，则显示每个 ARP 表的项。该指令的运行结果如图 5-1 所示，可以看到显示了 WLAN 接口和以太网的接口 ARP 表。

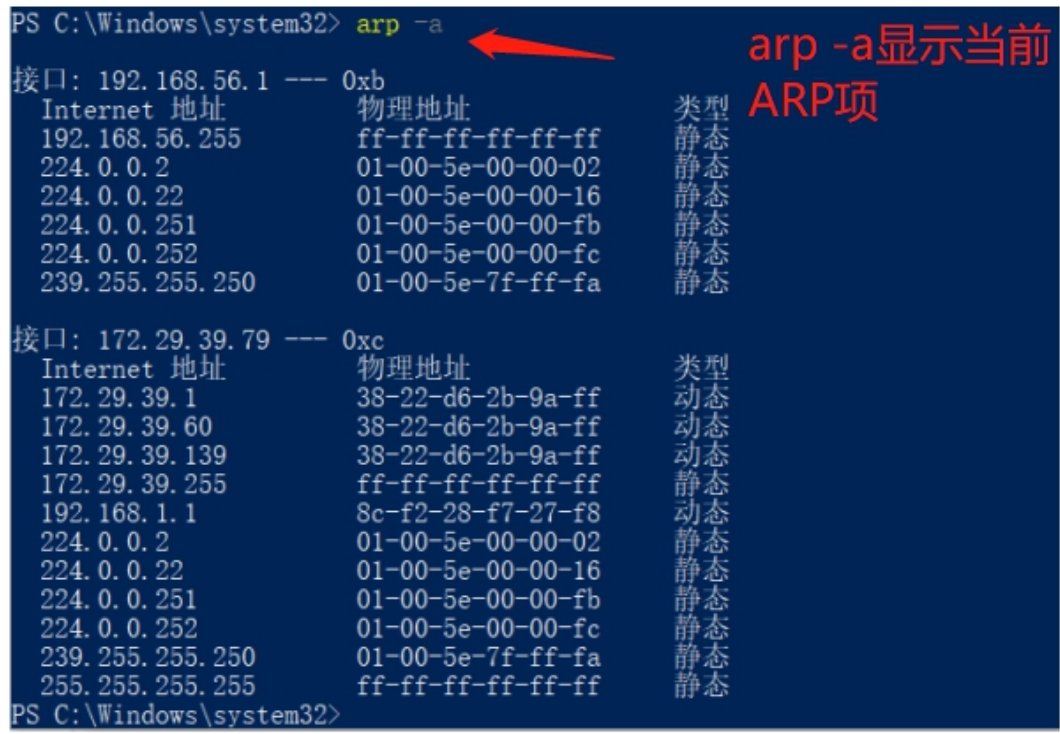


图 5-1 arp -a 的运行结果

5.2 arp -a inet_addr

如果有多个网卡，那么使用 arp -a 加上接口 IP 地址 inet_addr，就可以只显示与该接口相关的 ARP 缓存项目。例如对于图 5-1 的运行结果，Internet 地址选择接口 172.29.39.79 下面第一个地址 172.29.39.1，使用指令：arp -a 172.29.39.1 就可以只显示与该接口相关的 ARP 缓存项目，如图 5-2 所示。

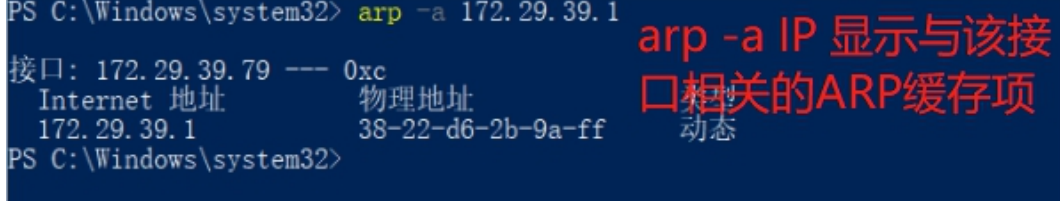


图 5-2 arp -a 172.29.39.1 的运行结果

5.3 arp -d inet_addr

这条指令可以删除 *inet_addr* 指定的主机对应的条目。记录下 5.2 中的 *Internet* 地址 192.168.56.255 以及其对应的物理地址 *ff-ff-ff-ff-ff-ff*, 使用指令 *arp -d 192.168.56.255* 删除对应条目, 再使用 *arp -a 192.168.56.255* 检查是否删除成功。运行结果如图 5-3 所示, 可以看到, *arp -d* 成功删除了指定的主机对应条目, 再次使用 *arp -a* 时无法找到该 *ARP* 项。

```
PS C:\Windows\system32> arp -d 192.168.56.255
PS C:\Windows\system32> arp -a 192.168.56.255
未找到 ARP 项。
```

未找到说明删除成功

图 5-3 *arp -d 192.168.56.255* 以及 *arp -a* 的运行结果

这里需要注意, 删除的条目应该找 **IP 类型为静态的条目**, 例如对 5.2 中的 172.29.39.1 动态条目进行删除, 删除指令后 *arp -a* 能够看到该条目还是存在的。删去的只有是静态的条目, 如图 5-2, 才能无法再找到 *ARP* 项。

5.4 arp -s inet_addr eth_addr

这条指令可以添加 *Internet* 地址 *inet_addr* 与物理地址 *eth_addr* 的关联条目。物理地址是用连字符分隔的 6 个十六进制字节。使用指令 *arp -s 192.168.56.255 ff-ff-ff-ff-ff-ff*, 再用 *arp -a 192.168.56.255* 检查, 运行结果如图 5-4 所示。可以看到, 之前删除的条目被成功添加回来。

```
PS C:\Windows\system32> arp -s 192.168.56.255 ff-ff-ff-ff-ff-ff
PS C:\Windows\system32> arp -a 192.168.56.255
```

接口: 192.168.56.1 --- 0xb		
Internet 地址	物理地址	类型
192.168.56.255	ff-ff-ff-ff-ff-ff	静态

添加成功

图 5-4 *arp -s 192.168.56.255 ff-ff-ff-ff-ff-ff* 以及 *arp -a* 的运行结果

这里 *ff-ff-ff-ff-ff-ff* 的物理地址在 5.3 中有记录, 实际上如果删除了一条 *Internet* 地址, 但是并未保存其物理地址的话, 就只能使用 *if_addr* 指定使用第一个适用的接口。

6 nslookup

nslookup 指令用于查询一台机器的 *IP* 地址对应的域名。该条指令查询需要手动 *quit* 退出, 运行结果如图 6 所示。

```
PS C:\Windows\system32> nslookup
默认服务器: ns.szptt.net.cn
Address: 202.96.134.133

> quit
PS C:\Windows\system32>
```

nslookup指令运行结果，使用quit退出

图 6 nslookup 指令的运行结果

7 route

route 指令用来操作网络路由表。

7.1 route print

route print 指令可以查看路由表，运行结果如图 7-1 所示。经过与之前指令的对比观察，该指令的运行结果与 netstat -r 完全一致，均为显示路由表。

在打印出来的结果 IPv4 的路由表里面有很多“在链路上”，英文“On-Link”，代表路由表的网关 IP 和 IF 参数对应的接口的 IP 是一样的情况。这种情况下就会在路由表显示“在链路上”。

```
PS C:\Windows\system32> route print

接口列表
11...0a 00 27 00 00 0b .....VirtualBox Host-Only Ethernet Adapter
6...60 f2 62 74 6b 1b .....Intel(R) Wireless-AC 9462
2...60 f2 62 74 6b 1c .....Microsoft Wi-Fi Direct Virtual Adapter
3...62 f2 62 74 6b 1b .....Microsoft Wi-Fi Direct Virtual Adapter #2
12...a8 5e 45 c0 e9 bc .....Realtek PCIe GbE Family Controller
1.....Software Loopback Interface 1

IPv4 路由表
活动路由:
网络目标      网络掩码      网关      接口      跃点数
0.0.0.0        0.0.0.0        0.0.0.0    172.29.39.1    35
127.0.0.0      255.0.0.0      255.0.0.0    在链路上      331
127.0.0.1      255.255.255.255    255.255.255.255    在链路上      331
127.255.255.255    255.255.255.255    255.255.255.255    在链路上      331
172.29.39.0      255.255.255.0      255.255.255.0      在链路上      291
172.29.39.79     255.255.255.255    255.255.255.255    在链路上      291
172.29.39.255    255.255.255.255    255.255.255.255    在链路上      291
172.30.255.2      255.255.255.255    255.255.255.255    在链路上      291
192.168.56.0      255.255.255.0      255.255.255.0      在链路上      36
192.168.56.1      255.255.255.255    255.255.255.255    在链路上      281
192.168.56.1      255.255.255.255    255.255.255.255    在链路上      281
192.168.56.255    255.255.255.255    255.255.255.255    在链路上      281
224.0.0.0        240.0.0.0        240.0.0.0      在链路上      331
224.0.0.0        240.0.0.0        240.0.0.0      在链路上      281
224.0.0.0        240.0.0.0        240.0.0.0      在链路上      291
255.255.255.255    255.255.255.255    255.255.255.255    在链路上      331
255.255.255.255    255.255.255.255    255.255.255.255    在链路上      281
255.255.255.255    255.255.255.255    255.255.255.255    在链路上      291

永久路由:
无
```

route print指令

路由表

```
IPv6 路由表
=====
活动路由:
接口 跃点数 网络目标      网关
12    291    ::/0                fe80::3a22:d6ff:fe2b:9aff
1     331    ::1/128             在链路上
12    291    2001:250:3c00:3509::/64 在链路上
12    291    2001:250:3c00:3509:3170:9862:78b7:cd71/128 在链路上
12    291    2001:250:3c00:3509:c078:724a:a230:cff7/128 在链路上
11    281    fe80::/64           在链路上
12    291    fe80::/64           在链路上
11    281    fe80::28a3:2492:c5e:ad73/128 在链路上
12    291    fe80::c078:724a:a230:cff7/128 在链路上
1     331    ff00::/8            在链路上
11    281    ff00::/8            在链路上
12    291    ff00::/8            在链路上
=====
永久路由:
无
PS C:\Windows\system32>
```

图 7-1 route print 指令的运行结果，与 netstat -r 指令运行结果相同

7.2 route delete inet_addr

该指令可以删除路由。其中 *inet_addr* 是“网络目标”IP 地址，选择路由表里的一条路由信息：127.255.255.255，其接口 172.29.39.1，使用指令 *route delete 127.255.255.255* 即可删除路由。指令运行结果如图 7-2 所示，可以看到再次使用 *route print* 查看时，已经没有 127.255.255.255。

```
PS C:\Windows\system32> route delete 127.255.255.255
操作完成!
PS C:\Windows\system32> route print
=====
接口列表
11...0a 00 27 00 00 0b .....VirtualBox Host-Only Ethernet Adapter
6...60 f2 62 74 6b 1b .....Intel(R) Wireless-AC 9462
2...60 f2 62 74 6b 1c .....Microsoft Wi-Fi Direct Virtual Adapter
3...62 f2 62 74 6b 1b .....Microsoft Wi-Fi Direct Virtual Adapter #2
12...a8 5e 45 c0 e9 bc .....Realtek PCIe GbE Family Controller
1.....Software Loopback Interface 1
=====
IPv4 路由表
127.255.255.255已经被删除
=====
活动路由:
网络目标      网络掩码      网关      接口      跃点数
0.0.0.0        0.0.0.0        0.0.0.0    172.29.39.1    172.29.39.79    35
127.0.0.0      255.0.0.0      在链路上    127.0.0.1      331
127.0.0.1      255.255.255.255 在链路上    127.0.0.1      331
172.29.39.0    255.255.255.0 在链路上    172.29.39.79    291
172.29.39.79   255.255.255.255 在链路上    172.29.39.79    291
172.29.39.255  255.255.255.255 在链路上    172.29.39.79    291
172.30.255.2   255.255.255.255 172.29.39.1    172.29.39.79    36
192.168.56.0   255.255.255.0 在链路上    192.168.56.1    281
192.168.56.1   255.255.255.255 在链路上    192.168.56.1    281
192.168.56.255 255.255.255.255 在链路上    192.168.56.1    281
224.0.0.0      240.0.0.0      在链路上    127.0.0.1      331
224.0.0.0      240.0.0.0      在链路上    192.168.56.1    281
224.0.0.0      240.0.0.0      在链路上    172.29.39.79    291
255.255.255.255 255.255.255.255 在链路上    127.0.0.1      331
255.255.255.255 255.255.255.255 在链路上    192.168.56.1    281
255.255.255.255 255.255.255.255 在链路上    172.29.39.79    291
=====
```

图 7-2 route delete 指令的运行结果，结果通过 route print 得到验证

7.3 route add inet_addr_1 inet_addr_2

该指令可以添加路由。其中, *inet_addr_1* 是网络目标 IP 地址, *inet_addr_2* 是网关地址。添加之前删除的 127.255.255.255, 使用指令 *route 127.255.255.255 172.29.39.1*, 指令的运行结果如图 7-3 所示。可以看到再次使用 *route print* 查看时, 127.255.255.255 已经被重新添加。

```
PS C:\Windows\system32> route add 127.255.255.255 172.29.39.1
操作完成!
PS C:\Windows\system32> route print
```

接口列表

```
11...0a 00 27 00 00 0b .....VirtualBox Host-Only Ethernet Adapter
6...60 f2 62 74 6b 1b .....Intel(R) Wireless-AC 9462
2...60 f2 62 74 6b 1c .....Microsoft Wi-Fi Direct Virtual Adapter
3...62 f2 62 74 6b 1b .....Microsoft Wi-Fi Direct Virtual Adapter #2
12...a8 5e 45 c0 e9 bc .....Realtek PCIe GbE Family Controller
1.....Software Loopback Interface 1
```

IPv4 路由表

重新添加路由

```
活动路由:
网络目标      网络掩码      网关      接口      跃点数
0.0.0.0        0.0.0.0        172.29.39.1  172.29.39.79  35
127.0.0.0      255.0.0.0      255.0.0.0    在链路上      331
127.0.0.1      255.255.255.255  255.255.255.255  在链路上      331
127.255.255.255 255.255.255.255 172.29.39.1    172.29.39.79  36
172.29.39.0    255.255.255.0    在链路上      172.29.39.79  291
172.29.39.79   255.255.255.255  在链路上      172.29.39.79  291
172.29.39.255  255.255.255.255  在链路上      172.29.39.79  291
172.30.255.2   255.255.255.255  172.29.39.1    172.29.39.79  36
192.168.56.0   255.255.255.0    在链路上      192.168.56.1  281
192.168.56.1   255.255.255.255  在链路上      192.168.56.1  281
192.168.56.255 255.255.255.255  在链路上      192.168.56.1  281
224.0.0.0      240.0.0.0        在链路上      127.0.0.1      331
224.0.0.0      240.0.0.0        在链路上      192.168.56.1  281
224.0.0.0      240.0.0.0        在链路上      172.29.39.79  291
255.255.255.255 255.255.255.255 在链路上      127.0.0.1      331
255.255.255.255 255.255.255.255 在链路上      192.168.56.1  281
255.255.255.255 255.255.255.255 在链路上      172.29.39.79  291
```

永久路由:
无

图 7-3 route add 指令的运行结果, 结果通过 route print 得到验证

实验结果:

实验结果与分析、结论:

(1) *ipconfig* 可用于显示当前的 *TCP/IP* 配置的设置值。这些值用来检验人工配置的 *TCP/IP* 设置是否正确。如果本地计算机和所在的局域网使用了动态主机配置协议, 通过 *ipconfig* 可以了解计算机是否成功租用到一个 IP 地址, 如果租用到则可以了解它目前分配到的的是什么地址。了解计算机当前 IP 地址、子网掩码和缺省网关实际上是进行测试和故障分析的必要项目。其中, 子网掩码是用于将主机 (*host*) 的 IP 地址分解成网络地址 (*network address*) 和主机地址 (*host address*) 的一组编码。形式上, 子网掩码是 32 位的, 比如 255.255.255.0, 一共是四个 8 位组, 即 11111111.11111111.11111111.00000000。

将这个子网掩码与主机的 *IP* 地址（192.168.16.187）进行二进制与运算，就可获得其网络地址，其结果就是 192.168.16.0。默认网关（*Default Gateway*）就是电脑要向网络上发送数据包时默认发送的地方。从网络架构来看，默认网关就是跟电脑在同一个网段里面（一般就是所在的子网），负责与其它网段（子网）进行通讯的那台设备。各指令运行截图保存为图 1-1——图 1-4。

（2）*ping* 是一个测试程序，用于确定本地主机是否能与另一台主机交换（发送与接收）数据报。如果 *ping* 运行正确，就可以排除网络访问层、网卡、*Modem* 的 *I/O* 线路、电缆和路由器等存在的故障。按缺省设置，运行 *ping* 命令时发送 4 个 *ICMP*（网间控制报文协议）“回送请求”，每个 32 字节数据；若正常应得到 4 个回送应答。*ping* 能够以毫秒为单位显示发送“回送请求”到返回“回送应答”之间的时间量。如果应答时间短，表示数据报不必通过太多的路由器或网络连接，速度比较快。*ping* 还能显示 *TTL*（*Time To Live* 存在时间值。通过 *TTL* 值推算数据包已经通过了多少个路由器：源地点 *TTL* 起始值（就是比返回 *TTL* 略大的一个 2 的乘方数）-返回时 *TTL* 值。以 *ping www.baidu.com* 指令为例，*baidu* 的 *IP* 地址实际为 14.215.177.39，由图 2-8 可以看到返回的 *TTL*=54，由此计算经过了 10 个路由器（64-54），最终到达接收方。注意传输方向为：本机<——14.215.177.39。各指令运行截图保存为图 2-1——图 2-8。

（3）*netstat* 用于显示与 *IP*、*TCP*、*UDP* 和 *ICMP* 协议相关的统计数据，用于检验本机各端口网络连接情况。*netstat* 的一些常用选项在实验过程中均有体现，各指令运行截图保存为图 3-1——图 3-5。

（4）*tracert* 命令可以用来跟踪数据报使用的路由，并列出在所经过的每个路由器上所花的时间。因此，*tracert* 一般用来检测故障的位置。其用法：只需在 *tracert* 后面跟一个 *IP* 地址或 *URL*，*tracert* 会进行相应的域名转换。关于 *ping* 和 *tracert* 指令的对比，对于相同的网站，可以看到 *ping* 的传输方向为本机<——被 *ping* 地址，*tracert* 的传输方向为本机——>*tracert* 的地址，在实验中，由图 2-4 可知，*ping www.baidu.com* 的 *TTL* 为 54，经过了 10 个路由器；此处 *tracert* 追踪经过了 11 个路由器，说明了两个方向经过的路由很可能不同。该命令运行截图保存为图 4-1，*ping* 与 *tracert* 的对比在 4.2 部分。

（5）*ARP* 用于确定对应 *IP* 地址的网卡物理地址。命令能够查看本地计算机或另一台计算机的 *ARP* 高速缓存中的当前内容。各指令运行截图保存为图 5-1——图 5-4。

（6）*nslookup* 命令的功能是查询一台机器的 *IP* 地址和其对应的域名，能监测网络中 *DNS* 服务器是否能正确实现域名解析它；它的运行需要一台域名

服务器来提供域名服务。该命令的一般格式为：*nslookup* [*IP* 地址/域名]。该指令的运行截图保存为图 6。

(7) *route* 用来显示、人工添加和修改路由表项目。大多数主机都驻留在只连接一台路由器的网段上。由于只有一台路由器，因此不存在使用哪一台路由器将数据报发表到远程计算机上去的问题，该路由器的 *IP* 地址可作为该网段上所有计算机的缺省网关来输入。但是，当网络上拥有两个或多个路由器时，可能想让某些远程 *IP* 地址通过某个特定的路由器来传递，而其他的远程 *IP* 则通过另一个路由器来传递。在这种情况下，必须人工将项目添加到路由器和主机上的路由表中。各指令运行截图保存为图 7-1——图 7-3。

实验小结：

实验的全部结果与指令的各种分析已经全部展示在上面的实验结果部分。在本次实验中，遇到的一些问题：

(1) 在了解 *ping* 指令的用法时，尝试使用自己的电脑 *ping* 同学的 *IP*，但是同样连接 *SZU_WLAN* 网络的情况下无法 *ping* 通。经过查阅资料，需要关闭电脑的防火墙，在实际操作中关闭了被 *ping* 电脑的防火墙后可以成功 *ping* 通。

(2) 实验中通过对 *ping* 和 *tracert* 的比较发现，*ping* 的传输方向和 *tracert* 的传输方向相反，经过 *TTL* 计算发现经过的路由器数目不同，也说明了两个方向经过的路由很可能不同。

(3) 在使用 *arp -a* 指令和 *arp -d* 指令时，后面的 *IP* 是指 *Internet* 地址，不是接口地址。这两个地址是不一样的，一开始输入接口地址时（即 172.29.39.1）时显示不存在，后面尝试接口地址下面的 *Internet* 地址才成功。

(4) 对于 *arp -d* 的删除指令，删除的条目应该找 *IP* 类型为静态的条目，例如对 5.1 中 172.29.39.1 动态条目进行删除，删除指令后 *arp -a* 能够看到该条目还是存在的。删去的只有是静态的条目，如图 5-2，才能得到无法再找到 *ARP* 项的预期结果。

本次实验的心得体会：通过本次实验，我学习并理解了 *ipconfig*、*ping*、*netstat*、*tracert*、*ARP*、*nslookup* 和 *route* 指令，对于这些指令具体的用途以及用法有了更深的理解，在实验结论部分也给出了很详细的总结。实验的一些指令如 *ping* 在我们的实际生活中应用也非常广泛，例如测试网络是否连通以及检测服务器响应速度。*ipconfig* 也可以很方便的看到自己的 *IP* 信息。目前唯一还有一点不太理解的是网上关于 *route* 指令看到的路由表里“在链路上”的解释，我之后还会继续查询资料。之后的实验我也会认真完成并注意细节。

指导教师批阅意见：

成绩评定：

指导教师签字：

年 月 日

备注：